



Energy and Performance Evaluation of Serverless and Serverful Models on Spark for Database Join Operations

Phan-An-Truong Tran, Laurent d'Orazio, Thuong-Cang Phan, Le Gruenwald

► To cite this version:

Phan-An-Truong Tran, Laurent d'Orazio, Thuong-Cang Phan, Le Gruenwald. Energy and Performance Evaluation of Serverless and Serverful Models on Spark for Database Join Operations. International Conference on Database and Expert Systems Applications (DEXA), Aug 2025, Bangkok, Thailand. hal-05136844

HAL Id: hal-05136844

<https://hal.science/hal-05136844v1>

Submitted on 30 Jun 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.

Energy and Performance Evaluation of Serverless and Serverful Models on Spark for Database Join Operations

Phan-An-Truong TRAN¹(✉), Laurent D’ORAZIO², Thuong-Cang PHAN³,
and Le GRUENWALD⁴

¹ Vinh Long University of Technology Education, Vietnam

`truongtpa@vlute.edu.vn`

² Univ Rennes, CNRS, IRISA

`laurent.dorazio@univ-rennes.fr`

³ Can Tho University, Vietnam

`ptcang@cit.ctu.edu.vn`

⁴ School of Computer Science, The University of Oklahoma, U.S.A

`ggruenwald@ou.edu`

Abstract. The demand for environmentally friendly cloud computing is on the rise, leading cloud service providers to focus on reducing carbon emissions by using renewable energy sources and energy-efficient computing models. This study assesses the performance and energy consumption of serverless and serverful architectures, specifically looking at join operations using Apache Spark for big data processing in a private cloud combined with Kubernetes. By using the TPC-DS benchmark, we examine the impact of cold-start and warm-start phases in the serverless environment, as well as the auto-scaling capabilities of Spark in serverless environments within the private cloud. The results show that the efficient and flexible resource management in serverless environments in private clouds leads to more optimal processing times and energy consumption compared to serverful architectures, especially in warm-start scenarios. These findings offer valuable insights for organizations seeking to streamline their big data infrastructure while also making a positive environmental impact within the IT industry.

Keywords: Serverless · Serverful · Warm-start · Cold-start · Energy consumption

1 INTRODUCTION

Big data processing has become essential for organizations to maximize value from information. The global big data analytics market, valued at \$307.51 billion in 2023, is projected to reach \$924.39 billion by 2032 with 13.0% CAGR [1]. In the field of big data analytics and processing, Apache Spark stands out as a powerful and flexible platform when compared with other tools such as Apache Hadoop, Flink, and Storm [8]. Spark’s advantages include in-memory data processing capabilities, support for diverse data types, and ease of application development.

Apache Spark can be deployed in both traditional on-premise environments and cloud services as virtual machines. However, operating Apache Spark in these environments faces significant limitations. In terms of cost, both on-premise and cloud-based solutions require substantial investments, especially when computational needs are intermittent but a certain number of virtual servers must be maintained. Additionally, Spark encounters difficulties in performance optimization due to unpredictable workloads. In this context, the serverless environment emerges as a potential solution, with advantages such as no need for server infrastructure management, simplified configuration, and a Pay-as-you-go pricing model. Services like Amazon EMR serverless, Google Cloud Dataproc serverless, and Azure HDInsight on Azure Kubernetes Service are leading this trend in big data processing with Apache Spark [5, 9]. Energy efficiency is increasingly important as data centers consume 1-2% of global electricity, driving cloud providers toward renewable energy adoption and carbon neutrality by 2050 [4]. However, there is a lack of comprehensive studies comparing performance and energy consumption between Apache Spark models in serverful (on-premise or virtual servers) and serverless environments. This creates difficulties for organizations in selecting the optimal environment for their specific needs.

To address this issue, we propose the study of Energy and Performance of serverless and serverful Models on Spark for Database Join Operations. This research makes the following contributions: (1) An analysis of factors influencing the performance of Apache Spark during join operations on TPC-H benchmark datasets of varying sizes. The analysis focuses on the impact of cold-start latency, worker node configurations, and resource scalability in both serverless and serverful deployment models; (2) A comparative study of performance and energy consumption between serverless and serverful environments during join-intensive workloads. The study highlights how deployment characteristics—such as the initialization state of serverless environments—affect execution behavior; and (3) An evaluation of the role of auto-scaling mechanisms in serverless environments, analyzing how dynamic resource allocation influences both performance and energy efficiency when processing large-scale data workloads with variable resource demands. The results of this study provide diverse insights for organizations considering transitioning or optimizing their big data processing infrastructure while contributing to the collective effort to minimize the environmental impact of the information technology industry.

The remainder of this paper is structured as follows. Section 2 reviews the related work on the performance and energy consumption of serverless and serverful architectures, particularly in the context of big data processing. Section 3 provides the background on the technical characteristics of Apache Spark in both serverless and serverful environments. Section 4 describes the monitoring system used for tracking performance and energy metrics. Section 5 presents the experimental setup and workloads, followed by detailed energy and performance evaluations in Section 6. Finally, Section 7 discusses the key findings from the experiments and provides suggestions for future research directions.

2 RELATED WORK

With the increase in big data, processing tools like Apache Hadoop and Apache Spark, along with deployment models such as serverless and serverful, are advancing rapidly. Some research [2, 16, 17] has been conducted to explore the performance evaluation and energy consumption of Spark in both the serverless and serverful environments. Changpeng et al. [17] evaluated the performance of Spark on a bare metal environment and a Kubernetes environment with several algorithms, such as SQL Join, Kmeans, and Wordcount and the study showed that Spark on Kubernetes performs better than the serverful environment in certain cases, such as memory-intensive stages (e.g., reduce stage in Sort), when using cache efficiently, or when applying custom task scheduling to improve data locality and executor utilization. Qi Zhang et al. [16] conducted a study comparing the performance of Spark when deployed on containers versus virtual machine environments. The research showed that Apache Spark’s performance on containers is higher than that of virtual machine environments. Alhindi et al. [2] conducted a study to evaluate the energy consumption of the serverless Function as a Service (FaaS) model on the OpenFaaS platform based on Docker and Kubernetes. The study proposed a method to estimate energy usage based on mathematical formulas of idle energy levels and execution energy levels. The OpenFaaS platform with FaaS was shown to have an energy level of 12-17.5% lower when compared to the Docker model.

Previous studies lack a comprehensive evaluation of executing Spark on a serverless model versus on a serverful model across factors like data size, nodes, and auto-scaling. Our study fills this gap by comparing the cold-start time and power consumption in the two serverless environments (cold-start and warm-start) and the serverful environment, and benchmarking Spark performance and energy efficiency across data sizes, cluster sizes, auto-scaling configurations, and serverless start states. Our key contributions include experimentally comparing Spark on serverless and serverful environments across multiple scenarios for database join operations. The results offer insights into performance differences, guiding appropriate architecture selections.

3 BACKGROUND

This section introduces the architecture, characteristics, advantages, and limitations of serverless and serverful systems in the context of executing join operators on Apache Spark. This section explains the differences in how to execute applications on serverless and serverful environments in terms of initialization, startup, processing data, and clean-up. We also explain how to measure energy consumption on Spark.

3.1 Serverless and Serverful environments

Serverful computing represents the traditional model where applications are deployed manually or semi-automatically on configured physical or virtual servers,

with developers and administrators responsible for provisioning, maintaining, and scaling the infrastructure [13]. In contrast, *serverless computing* is a cloud paradigm where providers manage all server infrastructures and automatically allocate computational resources, allowing developers to focus solely on application code [13]. The serverless model operates through stateless containers triggered by events, offering key advantages including on-demand computation, automatic scaling, and usage-based payment that enhance business flexibility and cost efficiency.

Within serverless environments, cold-starts and warm-starts significantly impact application performance [6]. A *cold-start* occurs when a container initializes for the first time or after extended inactivity, requiring complete runtime environment setup including container image retrieval, resource allocation, and application initialization, which introduces considerable latency. Conversely, *warm-starts* utilize recently active containers, dramatically reducing response time. This distinction between cold and warm starts substantively affects query response time and overall serverless application performance, particularly in scenarios with irregular workload patterns or sudden traffic spikes, making effective management of these states crucial for optimizing both performance and cost in serverless architectures.

3.2 Spark Serverless and Serverful

In the context of deploying Spark serverless, both public cloud and private cloud provide solutions that make application deployment easier for developers: Major public cloud providers such as AWS, Azure, and Google Cloud have developed specialized Spark serverless services like Amazon EMR serverless, Google Cloud Dataproc serverless, and Azure HDInsight on Azure Kubernetes Service [3]. These services offer automatic scaling capabilities, resource management, and pay-per-use billing based on actual usage. In private cloud serverless environments, Spark serverless deployment typically involves using Kubernetes (K8s) in combination with container management tools like Docker [10, 12, 15].

In this study, we chose a private cloud environment with the Kubernetes platform combined with the MinIO object storage system as the experimental environment for Spark on serverless due to their many similarities [7]. Figure 1 compares the differences between Spark applications when processing data in serverful and serverless environments. After a job is submitted to Spark, the scheduling begins before the application is executed. This process includes two main parts: DAG Scheduler and Task Scheduler. The DAG Scheduler is used to schedule and manage jobs as a directed, acyclic graph. This graph represents the dependencies between RDDs (Resilient Distributed Datasets) and transformations on them in a Spark application. The Task Scheduler manages and schedules tasks on executors on worker nodes in a Spark cluster. Its task is to distribute tasks to executors to perform parallel data processing. When the data processing is finished, the results are aggregated at the Spark Driver and returned to the user. Before termination, cleanup is performed and resources are freed [14] (Figure 1.a).

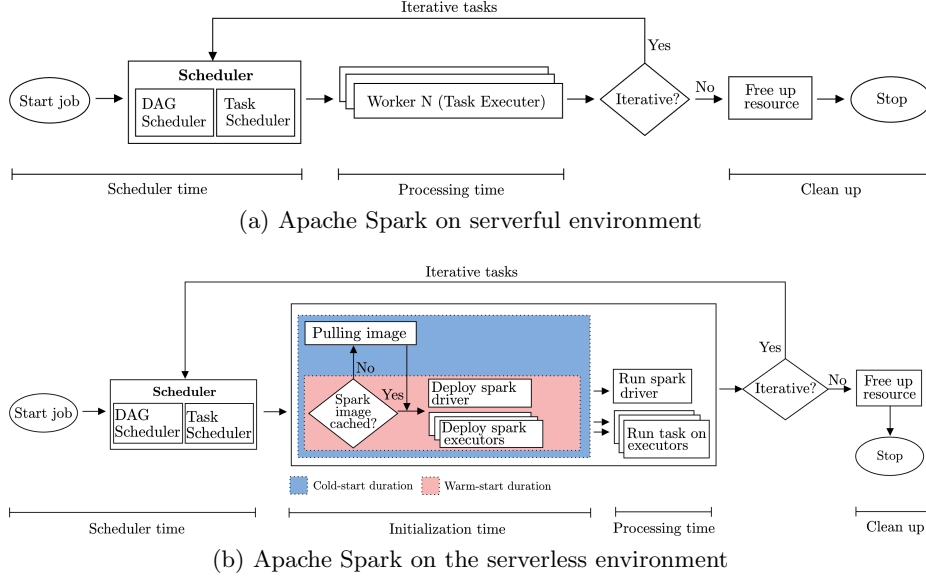


Fig. 1: Comparing Spark serverless and serverful environments

When comparing the processing of a Spark application on serverless and serverful environments, there are some differences in the initialization process, especially in the serverless environments with cold-start and warm-start phases. The cold-start phase in the serverless environment is the first stage when a Spark application is executed. During this phase, Spark has not yet initialized the environment. Therefore, it takes a specific time to do some work, such as downloading the Spark image to all workers and then initializing the *Spark driver* and *Spark workers* (Figure 1.b). After the environment has been initialized, the Spark application on the serverless environment begins to execute. From the second run onwards, the serverless environment is considered *warm-start* since Spark images already exist on the workers. It just needs to initialize the *Spark driver* and *Spark workers* and start executing the application. This phase is called *warm-start* (Figure 1.b).

Spark in serverless and serverful environments have different initialization characteristics. Spark in the serverless environment helps automate the initialization process, reclaim resources after each execution to help save costs, and auto-scale. Meanwhile, Spark must manually initialize and reclaim resources in the serverful environment.

4 PROPOSED MONITORING SYSTEM

Monitoring the performance and energy consumption of Apache Spark in both serverless and serverful environments presents a major challenge in private cloud

settings due to the complexity of multi-server hardware and distributed processing. To address this, we developed GUESS [11], a monitoring system that collects key metrics such as resource utilization, query execution time, and power consumption using Prometheus, Grafana, Spark History Server, and OpenManage Enterprise Power Manager. Figure 2 illustrates the *GUESS* model, designed to track Spark’s performance and energy usage across different deployment environments.

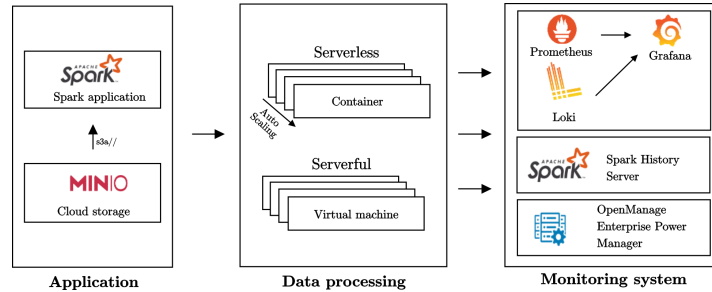


Fig. 2: GUESS: Monitoring Join query Execution in serverless and serverful Spark [11]

Energy consumption is measured at the hardware level using Dell iDRAC with 1-second granularity. Performance metrics including CPU, memory, I/O, and network utilization are collected through Prometheus and visualized via Grafana dashboards [11]. Total energy is calculated by aggregating power usage across all cluster nodes during the complete execution period from job submission to cleanup, enabling accurate comparison between serverless and serverful environments.

5 EXPERIMENTAL SETUP AND WORKLOADS

In this section, we outline the experimental environment and workloads utilized to assess the performance and energy consumption of Apache Spark during join operations in both serverless and serverful architectures.

5.1 Experimental setup

Figure 3 illustrates the proposed experimental architecture, deployed on a private cloud infrastructure consisting of 5 physical Dell PowerEdge servers, each hosting 7 virtual machines running Ubuntu Server 22.04. Each VM is allocated 6vCPU, 16GB memory and 128GB storage, connected via 10Gbps Ethernet network. Network bandwidth and latency are monitored to ensure consistent performance across experiments. In serverful environments, Spark is installed manually on each virtual machine. In the serverless environment, Spark is deployed on a

Kubernetes cluster in the private cloud with the horizontal pod autoscaling (HPA) mode, which is the typical environment we use in experiments due to its similarities with Spark serverless on public cloud services. The HPA is configured with 70% CPU target utilization, scaling 2 pods per 30 seconds (up) and 1 pod per 60 seconds (down), with 5-20 executor range. Cold-start time is measured from pod creation to first task execution, while warm-start time begins from executor reuse to task execution, tracked through Kubernetes events and Spark logs. Using the MinIO cluster for storage as an object storage saves the data produced by our experiments.

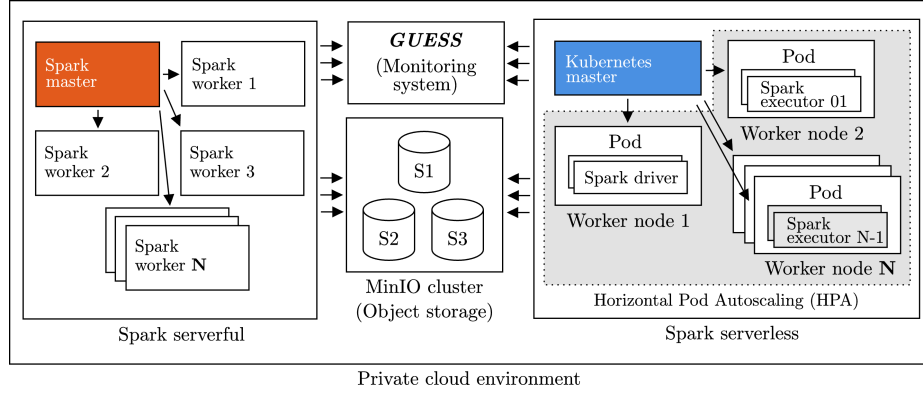


Fig. 3: The proposed experimental architecture

5.2 Database size

To evaluate Spark performance across serverless and serverful environments, we employed the TPC-DS database benchmark, a sophisticated workload specifically designed for assessing big data processing capabilities. This benchmark is particularly valuable for our study due to its complex query patterns across 24 database tables, diverse analytical scenarios represented by 99 distinct query types, and extensive join operations that effectively stress distributed processing frameworks. Our experiments consisted of three database scenarios: *Case Study 1 (small database - 10GB)*: Examines initialization delay from cold-start in serverless environments using a small dataset where startup time dominates execution time, isolating container image pulling and environment setup impacts; *Case Study 2 (medium database - 25GB)*: Evaluates performance and energy consumption across all three deployment environments (serverful, serverless cold-start, and serverless warm-start) using a balanced workload that allows comparison of processing efficiency without overwhelming infrastructure constraints; and *Case Study 3 (large database - 50GB)*: Stressed automatic scaling capabilities with our largest database (50GB), allowing serverless environments

to scale between 10-20 nodes dynamically. While 50GB may appear modest for 10-20 nodes, this size was chosen to demonstrate auto-scaling behavior under controlled conditions and to ensure completion within reasonable experimental timeframes. Future work will examine larger datasets up to multi-terabyte scales.

Each environment maintained consistent base configurations (12GB driver memory, 6 executor cores) with the primary distinction being the ImagePullPolicy setting between serverless cold-start (*Never*) and warm-start (*Always*). For the heavy workload scenario, we enabled dynamic allocation in serverless environments (max: 20 executors, min: 5 executors), a capability not available in the serverful environment.

6 Energy and Performance Evaluation

In this section, we present the results of our experiments, focusing on the energy consumption and performance of Apache Spark in both serverless and serverful environments.

6.1 Case Study 1: The Impact of "Cold-start" on Join Query Performance

Our Case Study 1 aims to quantify the time duration of the cold-start process in the serverless environment compared to the serverful environment. In the experiment, we use the *PullPolicy* parameter in Kubernetes to control how container images are pulled. Specifically, we compare two values of *PullPolicy*: *Never* (corresponding to the cold-start serverless environment) and *Always* (corresponding to the warm-start serverless environment). With *Never*, the container image is only pulled if it does not exist locally, while *Always* forces Kubernetes to pull a new image from the registry every time a container is started. By changing *PullPolicy*, we can evaluate the impact of the image-pulling process on the cold-start time in the serverless environment. The Spark images are stored on *Docker Hub* with different tags (image digests) to allow for multiple tests.

The experiment is conducted on a compute cluster consisting of 1 master node and 10 worker nodes. Each node is configured with 12GB of memory, 6 vCPU executor cores, and 10 worker nodes for each environment (serverful, cold-start serverless, and warm-start serverless). We use the TPC-DS benchmark with a database size of 10GB and execute a total of 31 different queries, with each environment being tested 5 times consecutively to obtain the most accurate results.

The results in Table 1 show that the serverless environment in the cold-start state the serverful environment, while in the warm-start state, the serverless environment has 4.8% higher total average processing time than the serverful environment. The reason for this difference lies in the cold-start phase of the serverless model, where the startup process requires more preparation steps than the serverful model, leading to longer startup times. Specifically, when a serverless cluster begins processing, the workers may need to download container

Table 1: Average processing time (Seconds) by each stage in Case Study 1

Stage	Serverless cold-start	Serverless warm-start	Serverful
Stage 1: Initialization			
Scheduling	3.3	3.3	2.0
Pulling images	35.5	0.0	0.0
Creating	11.0	11.0	0.0
Stage 2: Processing			
Running the app	382.7	374.2	370.0
Stage 3: Clean-up			
Killing	2.5	2.5	1.0
Total Time	435.0	391.0	373.0

Note: Total time is calculated as the sum of Stage 1 (Initialization), Stage 2 (Processing), and Stage 3 (Clean-up).

images from a registry (such as Docker Hub or Amazon ECR) to set up the execution environment. However, this process does not occur synchronously and is influenced by several factors, such as network bandwidth, disk read/write speed, and other issues.

For Apache Spark in a serverless environment, when the workers cannot pull images from the registry synchronously, some workers have to wait until all others complete initialization before starting the tasks, directly affecting the performance of distributed operations, especially complex operations like Join. A Join operation requires the simultaneous distribution and processing of data across workers, but if some workers are not ready, the process will be delayed, increasing latency and causing imbalance in the computation. Furthermore, Spark can encounter data skew issues, where some workers have to process significantly more data than others, leading to bottlenecks and reduced overall cluster performance. Figures 4.a and 4.b illustrate that the read and write latencies of the serverful environment (Triangle points) are lower than those of serverless (Circle and Square points). To mitigate these negative impacts, solutions such as warm-start, caching optimization, and the use of *Adaptive Query Execution (AQE)* can be applied to improve performance and reduce the latency caused by cold-start.

6.2 Case Study 2: Comparing the Performance and Energy Consumption

In this experiment, we evaluate the performance and energy consumption of query operations in Spark in three environments: serverless with cold-start, serverless with warm-start, and serverful, using a TPC-DS database with a size of 25GB. The experiment was conducted with 10 worker nodes to compare data processing capabilities and resource optimization across these environments, aiming

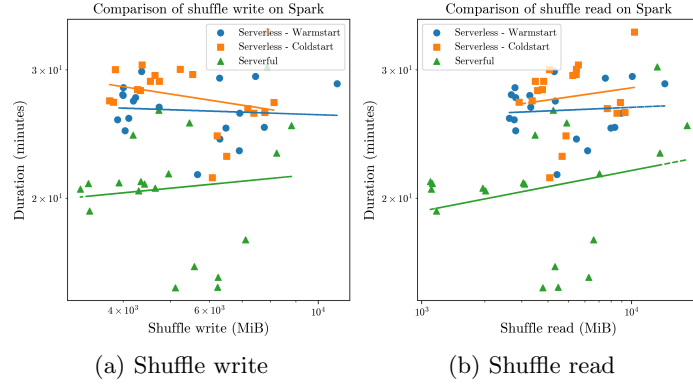


Fig. 4: Comparison of the data shuffling process between serverless and serverful environments

to identify differences in performance and energy consumption in various deployment scenarios. We used 31 sample queries, primarily focusing on data Join operations. Each query in every environment was executed 5 times, and the experimental results were averaged over the 5 runs.

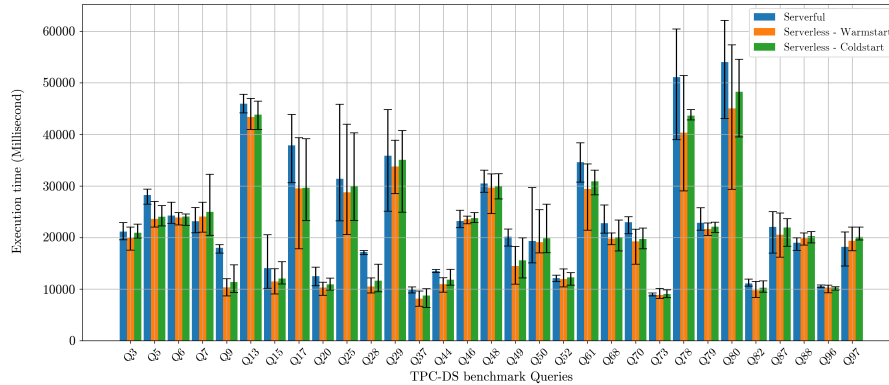


Fig. 5: TPC-DS benchmark queries on dataset of size is 25GB

Analyzing the error bars in the average execution time chart of 31 TPC-DS queries (Figure 5) reveals that a serverless environment has higher stability than the serverful environment. The shorter error bars in the serverless environment indicate lower fluctuations and better predictability. This stability is due to the serverless environment's isolation and flexible resource allocation capabilities, which minimize resource contention between queries. However, for some specific

Energy and Performance Evaluation of Serverless and Serverful Models

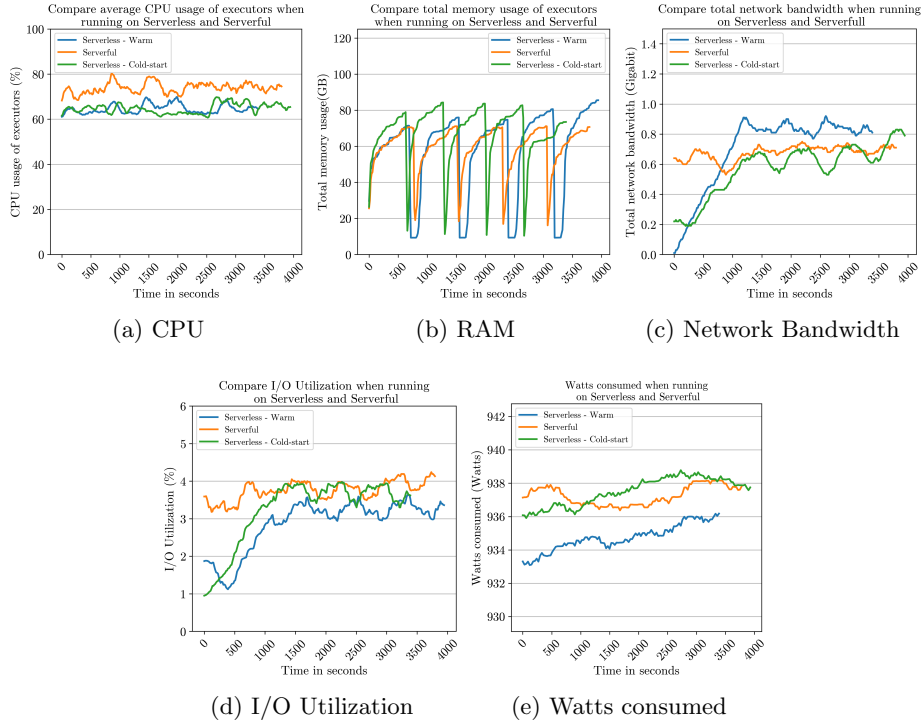


Fig. 6: compares performance and energy consumption in Case Study 2

queries, such as $q78$ and $q80$, the error bars on the serverless environment are longer, suggesting that individual query characteristics also affect stability. The results show that the serverless environments perform better than the serverful environment for most of the 31 TPC-DS queries, particularly with the warm-start serverless environment. Figure 6 comparing the performance and energy consumption and Table 2 shows the average execution time across 5 consecutive runs, which reveals that the warm-start serverless environment has the lowest execution time (651.90 seconds), followed by the cold-start serverless environment (677.49 seconds), and the serverful environment (736.83 seconds). The serverless environment also demonstrates better energy consumption than the serverful environment. The warm-start serverless environment consumes the least energy (1200 Wh), followed by the cold-start serverless environment (1380 Wh), and the serverful environment (1330 Wh).

The efficiency in performance and energy consumption of the serverless environment, especially in the warm-start scenario, can be explained by several factors. First, the serverless architecture allows for flexible resource allocation and scaling, ensuring that resources are optimally used for each query, particularly in complex *join* operations within the TPC-DS database. Second, the initial-

Table 2: Comparison of execution time and energy consumption in Case Study 2

Environment	Execution time (s)	Total energy usage (Wh)
Serverless cold-start	677.49	1380
Serverless warm-start	651.90	1200
Serverful	736.83	1330

ization process in the serverless environment, including container creation and resource allocation, is optimized to minimize overhead, especially in warm-start scenarios. This reduces waiting time and allows the system to quickly become ready to handle workloads. Moreover, the nature of the TPC-DS workload, with short-lived join queries and frequent changes in resource requirements, aligns well with the flexible scaling capabilities of serverless. In join queries, *data locality* plays a crucial role: serverless intelligent resource scheduling ensures that tasks are processed closer to the data, reducing the need for network data transfer and thus improving overall performance. In contrast, serverful environments with fixed resources can lead to resource underutilization during low demand and resource contention during peak loads, negatively impacting both performance and energy efficiency. Serverful environments often fail to optimize *data locality* and cannot dynamically adjust resources, resulting in lower processing efficiency, especially in large-scale join operations such as those in TPC-DS.

6.3 Case Study 3: Impact of Auto-Scaling of Resources

In this experiment, our focus is to study the impact of the automatic scaling capabilities of the serverless environment when processing TPC-DS queries with a larger database size of 50 GB. We evaluate the effectiveness of automatically adjusting the number of worker nodes in a serverless environment based on the workload. We compare the resulting query processing performance and energy consumption with those in a serverful environment. The experiment was conducted on a compute cluster consisting of 1 master node and a varying number of worker nodes ranging from 10 to 20 nodes. For the serverless environment, we use the configured auto-scaling parameters `spark.dynamicAllocation.enabled`, `spark.dynamicAllocation.maxExecutors`, and `spark.dynamicAllocation.minExecutors` to allow Spark to automatically adjust the number of executors based on resource demand. We measured and compared the cold-start time, total processing time, and power consumption in the serverless environment (in both cold-start and warm-start states) and the serverful environment.

When we examine the cold-start time results in Table 3, we can see that enabling auto-scaling actually increases the cold-start time in both serverless cold-start and warm-start states. Specifically, with a configuration of 1 master node and 10 worker nodes, the cold-start time increases from 87.50 seconds to

98.50 seconds for the cold-start serverless environment, and from 19.50 seconds to 30.50 seconds for the warm-start serverless environment when auto-scaling is enabled. We see a similar trend with the configuration of 1 master node and 20 worker nodes. This increase occurs because of the initialization and configuration processes of the components relating to auto-scaling, such as the Kubernetes Cluster Autoscaler and Horizontal Pod Autoscaler, which require additional time for setup and synchronization. However, in the serverful environment with fixed resource allocation, enabling or disabling auto-scaling has no effect on the cold-start time. Even though enabling auto-scaling increases the cold-start time in the serverless environment, it can be thought of as an acceptable trade-off to achieve flexible scalability and more efficient resource utilization while processing queries on big data.

Table 3: Comparing serverless and serverful performance with auto-scaling in Case Study 3

	1 master, 10 workers			1 master, 20 workers		
	Cold-start	Warm-start	Serverful	Cold-start	Warm-start	Serverful
I. Without Auto-Scaling						
Cold-start time	87.5	15.0	2.0	150.0	30.0	2.0
Processing time	1134.6	1146.0	1150.0	1024.8	960.0	1025.0
Total (s)	1222.1	1161.0	1152.0	1174.8	990.0	1027.0
Power (Wh)	1671	1600	1650	2500	2250	2450
II. With Auto-Scaling						
Cold-start time	98.5	15.0	2.0	150.0	30.0	2.0
Processing time	1100.0	1140.0	1150.0	990.0	951.0	1025.0
Total (s)	1198.5	1155.0	1152.0	1140.0	981.0	1027.0
Power (Wh)	1655	1569	1650	2468	2187	2450

For the processing time in Table 3, enabling auto-scaling significantly improves both the serverless environments with cold-start and warm-start states. With a configuration of 1 master node and 10 worker nodes, the processing time slightly decreases from 1134.60 seconds to 1100.00 seconds for the cold-start serverless environment and from 1146.00 seconds to 1140.00 seconds for the warm-start serverless environment when auto-scaling is applied.

When comparing power consumption in Table 3, it is evident that applying auto-scaling results in the use of energy more efficiently, especially in the serverless environments. With a configuration of 1 master node and 10 worker nodes, the power consumption decreased from 1600 Wh to 1569 Wh (a 1.97% reduction). Similarly, with a configuration of 1 master node and 20 worker nodes, enabling auto-scaling reduced power consumption from 2250 Wh to 2187 Wh (a 2.88% reduction) in the warm-start serverless scenario. These results highlight the role of auto-scaling in optimizing resource utilization and minimizing energy waste. In contrast, power consumption in the serverful environment showed no difference between applying and not applying auto-scaling (with energy con-

sumption being 4.90% and 10.73% higher compared to warm-start serverless, respectively), indicating that serverful architectures may be less flexible in optimizing resources and saving energy compared to serverless environments.

Although the energy savings may seem small, at large-scale data centers, auto-scaling can lead to significant reductions in energy consumption. This is crucial for zero-carbon cloud computing, where providers aim to cut emissions through renewable energy and efficient computing models. As data and computational demands grow, auto-scaling and serverless architectures will be essential for sustainable cloud computing.

7 CONCLUSION AND FUTURE RESEARCH

This study highlights the fundamental differences between serverless and serverful environments in terms of performance, energy consumption, and resource management. serverless architectures excel in handling intermittent and unpredictable workloads by enabling automatic initialization, elastic scaling, and timely resource deallocation, which contributes to greater energy efficiency. In contrast, serverful models offer consistent performance and lower latency for continuous workloads but are generally less efficient due to static resource allocation. Our experimental findings show that, under warm-start conditions, serverless systems outperform serverful setups in both execution speed and energy usage. However, cold-start latency remains a critical drawback for serverless deployments, limiting their suitability for real-time applications. Overall, the decision to adopt a serverless or serverful architecture depends on multiple factors, including workload pattern, data volume, operational costs, and system management capabilities.

Future studies should further explore the applicability of serverless Spark in public cloud ecosystems such as AWS Lambda, Google Cloud Functions, and Azure Functions, where service constraints and runtime environments vary. A deeper investigation into cold-start mitigation techniques—especially in relation to interarrival times—could provide key strategies for improving responsiveness. Moreover, examining hybrid deployment models that integrate both serverless and serverful paradigms may yield solutions that balance flexibility and performance. Evaluating energy efficiency, cost trade-offs, and scalability in real-world cloud environments will also help to establish practical best practices for deploying Apache Spark in production workloads.

References

1. Big data analytics market forecast, 2024–2032 (2024), <https://www.fortunebusinessinsights.com/big-data-analytics-market-106179>, accessed: 2025-03-25
2. Alhindi, A., Djemame, K., Heravan, F.B.: On the power consumption of serverless functions: An evaluation of openfaas. In: IEEE International Conference on Utility and Cloud Computing (UCC) (2022). <https://doi.org/10.1109/UCC56403.2022.00064>

3. Borra, P.: Serverless computing: The future of scalability and efficiency with aws, azure, and gcp. *International Journal of Advanced Research in Science Communication and Technology* (2025). <https://doi.org/10.48175/IJARSCT-23373>
4. Cao, Z., Zhou, X., Hu, H., Wang, Z., Wen, Y.: Toward a Systematic Survey for Carbon Neutral Data Centers. *IEEE Communications Surveys & Tutorials* (2022). <https://doi.org/10.1109/COMST.2022.3161275>
5. Darius, P.S., et al.: From Data to Insights: A Review of Cloud-Based Big Data Tools and Technologies. *Big Data Computing: Advances in Technologies, Methodologies, and Applications* (2024). <https://doi.org/10.11648/j.ajist.20240803.11>
6. Ghorbian, M., Ghobaei-Arani, M.: A survey on the cold start latency approaches in serverless computing: an optimization-based perspective. *Computing* (2024). <https://doi.org/10.1007/s00607-024-01335-5>
7. Govind, H., González-Vélez, H.: Benchmarking serverless workloads on kubernetes. In: *IEEE/ACM International Symposium on Cluster Computing and the Grid (CCGRID)* (2021). <https://doi.org/10.1109/CCGrid51090.2021.00085>
8. Luengo, J., García-Gil, D., Ramírez-Gallego, S., García, S., Herrera, F.: Big Data: Technologies and Tools. *Big Data Preprocessing* (2020). https://doi.org/10.1007/978-3-030-39105-8_2
9. Mittal, V., Qi, S., Bhattacharya, R., Lyu, X., Li, J., Kulkarni, S.G., Li, D., Hwang, J., Ramakrishnan, K.K., Wood, T.: Mu: An efficient, fair and responsive serverless framework for resource-constrained edge clouds. In: *SoCC '21: Proceedings of the ACM Symposium on Cloud Computing* (2021). <https://doi.org/10.1145/3472883.3487014>
10. Perez, A., Risco, S., Naranjo, D.M., Caballer, M., Molto, G.: On-premises serverless computing for event-driven data processing applications (2019). <https://doi.org/10.1109/CLOUD.2019.00073>
11. Phan, A.T., d'Orazio, L., Phan, T., Gruenwald, L.: GUESS: monitoring join query execution in serverless and serverful spark. In: *International Conference on Database Systems for Advanced Applications (DASFAA)*, Gifu, Japan, July 2024 (2024). https://doi.org/10.1007/978-981-97-5575-2_45
12. Sethy, K.K., Singh, D., Biswal, A.K., Sahoo, S.: Serverless Implementation of Data Wizard Application using Azure Kubernetes Service and Docker. In: *1st IEEE International Conference on Industrial Electronics: Developments Applications (ICIDEA)* (2022). <https://doi.org/10.1109/ICIDEA53933.2022.9970103>
13. Shafiei, H., Khonsari, A., Mousavi, P.: Serverless computing: A survey of opportunities, challenges, and applications. *ACM Computing Surveys* (2022). <https://doi.org/10.1145/3510611>
14. Shanahan, J.G., Dai, L.: Large scale distributed data science using apache spark. In: *21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (2015). <https://doi.org/10.1145/2783258.2789993>
15. Vahidinia, P., Farahani, B., Aliee, F.S.: Cold start in serverless computing: Current trends and mitigation strategies. In: *International Conference on Omni-layer Intelligent Systems (COINS)* (2020). <https://doi.org/10.1109/COINS49042.2020.9191377>
16. Zhang, Q., Liu, L., Pu, C., Dou, Q., Wu, L., Zhou, W.: A comparative study of containers and virtual machines in big data environment. In: *IEEE International Conference on Cloud Computing, CLOUD* (2018). <https://doi.org/10.1109/CLOUD.2018.00030>
17. Zhu, C., Han, B., Zhao, Y.: A Comparative Study of Spark on the bare metal and Kubernetes. In: *International Conference on Big Data and Information Analytics (BigDIA)* (2022). <https://doi.org/10.1109/BigDIA51454.2020.00027>